Modelado algorítmico de la generación hidroeléctrica del Sistema Interconectado Nacional durante el Fenómeno de El Niño: Fase 2

[Nombre completo del estudiante 1]
[Nombre completo del estudiante 2]
[Nombre completo del estudiante 3]
Departamento de Energía, Universidad de la Costa (CUC)
Programa de Ingeniería Eléctrica
[Código y nombre de la asignatura]
[Nombre del docente]
16 de octubre de 2026

Fase 2: Desarrollo progresivo con condicionales y ciclos

Contexto y alcance

El sistema eléctrico colombiano depende principalmente de las centrales hidroeléctricas, por lo que su confiabilidad baja durante el Fenómeno de El Niño (Restrepo-Trujillo et al., 2022). En esos años llueve menos y bajan los caudales de los ríos (Poveda et al., 2001; Gómez-Dueñas et al., 2024), lo que pone en riesgo la energía que el sistema puede garantizar (Cramton y Stoft, 2007; Moreno Chuquen y Cantillo Luna, 2020). En la Fase 1 se diseñó un algoritmo que calcula la energía que pueden generar varias centrales en un mes con la fórmula de potencia hidráulica $P = 9{,}81 \cdot Q \cdot H \cdot \eta / 1\,000$ (en MW) (Sritram y Suntivarakorn, 2017) y clasifica el riesgo de que falte energía.

En esta Fase 2 el algoritmo se convierte en un programa de Python con un menú de cinco opciones: registrar los datos generales, registrar centrales, consultar una central, generar un reporte y salir. El programa también indica el estado de cada central para detectar instalaciones con problemas, que es el aporte del ingeniero eléctrico frente al eje problematizador, y avisa cuándo se necesita apoyo de plantas térmicas (Medina y Moreno, 2007). No se incluyen el manejo de archivos, el despacho económico ni la simulación de caudales, temas que requieren modelos más avanzados (Hoseinzadeh et al., 2020; Tayyeh y Mohammed, 2024).

Atención a las observaciones de la Fase 1

Tabla 1
Observaciones de la Fase 1 y ajustes realizados

Observación	Ajuste realizado
Dos referencias (CREG y XM) no venían de bases de datos institucionales.	Se cambiaron por artículos de IEEE Xplore y Scopus (Cramton y Stoft, 2007; Restrepo-Trujillo et al., 2022).
El indicador de autonomía daba resultados contradictorios y el balance de volúmenes era difícil de explicar.	Se simplificó el modelo: el nivel del embalse se ingresa directamente en porcentaje y se retiró la autonomía.
No se probó la validación de los datos de cada central.	Se agregaron los casos V-08 a V-10 de la Tabla 8.
Las horas máximas eran fijas (744 h).	Ahora el máximo depende de los días del mes (días × 24).
[Observación de la docente pendiente por transcribir]	[Ajuste]

Nota. Elaboración propia.

Variables y restricciones

Tabla 2
Variables del programa

Variable	Tipo Python (PSeInt)	Unidad	Restricción
opcion	int (Entero)	—	1 a 5
region, nombre, buscado	str (Caracter)	—	Texto
dias	int (Entero)	días	28 a 31

Variable	Tipo Python (PSeInt)	Unidad	Restricción
horas	int (Entero)	h	1 a dias × 24
nivel	float (Real)	%	0 a 100
caudal_medio, demanda	float (Real)	m³/s, GWh	Mayor que 0
caudal, altura, p_nominal	float (Real)	m³/s, m, MW	Mayor que 0
eficiencia	float (Real)	—	Mayor que 0 y hasta 1
p_bruta, p_efectiva, energia, factor	float (Real)	MW, GWh, %	Se calculan
estado, riesgo, alerta	str (Caracter)	—	Valores fijos (Tablas 6 y 7)
datos_listos, repetido, encontrada, nino	bool (Lógico)	—	True o False
nombres, energias, estados, caudales...	list (arreglo)	—	Máximo 50 elementos
energia_total, caudal_total, con_problema	float / int (acumuladores)	GWh, m³/s	Empiezan en 0

Nota. Las listas guardan los datos de todas las centrales; la posición i de cada lista corresponde a la misma central. Elaboración propia.

Tabla IPO

Tabla 3

Tabla IPO por opción del menú

Opción	Entrada	Proceso	Salida
1. Datos generales	Región, días, horas, nivel, caudal medio, demanda	Validar cada dato con while	Datos guardados
2. Registrar central	Nombre, caudal, altura, eficiencia, potencia nominal	Validar; buscar repetidos con for; calcular potencia, energía, factor de planta y estado	Central guardada en las listas
3. Consultar	Nombre de la central	Buscar con for	Potencia, energía, factor y estado
4. Reporte	Listas de centrales	Sumar con for; calcular cobertura, déficit e índice de aportes; clasificar riesgo y alerta	Reporte de la región
5. Salir	—	Terminar el ciclo while	Fin del programa

Nota. Elaboración propia.

Reglas de negocio

Tabla 4

Reglas de negocio

Código	Regla
RN-01	Se pueden registrar máximo 50 centrales.
RN-02	Los datos generales se registran antes que las centrales.
RN-03	Días entre 28 y 31; horas entre 1 y días × 24; nivel entre 0 y 100 %.
RN-04	Caudal, altura, potencia nominal, caudal medio y demanda deben ser mayores que 0; la eficiencia, mayor que 0 y hasta 1.
RN-05	No se repiten nombres de centrales.
RN-06	La potencia efectiva no puede superar la potencia nominal.
RN-07	Estado de la central: LIMITADA si la potencia bruta supera la nominal; SUBUTILIZADA si el factor de planta es menor que 30 %; EFICIENCIA BAJA si la eficiencia es menor que 0,85; si no, ADECUADA.
RN-08	Hay condición El Niño si el índice de aportes (caudal total / caudal medio) es menor que 0,80.
RN-09	Riesgo: NORMAL si nivel > 70 % y cobertura ≥ 100 %; SEGUIMIENTO si nivel > 50 %; ALERTA si nivel > 30 %; si no, CRÍTICO.
RN-10	Alerta: ACTIVAR RESPALDO TÉRMICO si el riesgo es CRÍTICO o hay déficit con El Niño; VIGILANCIA si hay solo déficit o solo El Niño; si no, OPERACIÓN NORMAL.

Nota. Los umbrales de RN-07 son valores definidos por el grupo con fines didácticos. Elaboración propia.

Condicionales y ciclos utilizados

Los estudiantes que empiezan a programar suelen tener más dificultades con las condicionales y los ciclos (Qian y Lehman, 2017), que son la base del pensamiento computacional en ingeniería (Ehsan et al., 2020). Por eso el programa usa solo las estructuras básicas de la Tabla 5. Se eligió Python porque su sintaxis es clara y permite concentrarse en la lógica (Fangohr, 2004; Kunkle y Allen, 2016).

Tabla 5

Estructuras de control del programa

Estructura	Python (PSeInt)	Dónde se usa
Condicional simple	if (Si)	Detectar una central repetida
Condicional doble	if-else (Si-SiNo)	Limitar la potencia; calcular el déficit
Condicional múltiple	if-elif-else (Segun / Si anidado)	Menú, estado de la central y riesgo
Condicional compuesta	and, or (Y, O)	Validaciones y alerta
Ciclo while	while (Mientras)	Repetir el menú y volver a pedir un dato inválido
Ciclo for	for (Para)	Buscar repetidos, consultar y sumar en el reporte

Nota. Elaboración propia.

Matriz de decisiones

Tabla 6

Matriz de decisiones del estado de la central y del riesgo

Condición	Resultado	Caso
Potencia bruta > potencia nominal	LIMITADA	CH-1 y CH-2, Esc. 1
Factor de planta < 30 %	SUBUTILIZADA	Esc. 4
Eficiencia < 0,85	EFICIENCIA BAJA	CH-3, Esc. 5
Ninguna de las anteriores	ADECUADA	Esc. 2
Nivel > 70 % y cobertura ≥ 100 %	Riesgo NORMAL	Esc. 1
Nivel > 50 % (sin lo anterior)	Riesgo SEGUIMIENTO	Esc. 2 y 5
Nivel entre 30 % y 50 %	Riesgo ALERTA	Esc. 3
Nivel ≤ 30 %	Riesgo CRÍTICO	Esc. 4

Nota. Las condiciones del estado se revisan en orden: si una se cumple, ya no se revisan las siguientes. Elaboración propia.

Tabla 7

Matriz de decisiones de la alerta

¿Riesgo CRÍTICO?	¿Déficit?	¿El Niño?	Alerta	Caso
No	No	No	OPERACIÓN NORMAL	Esc. 1
No	Sí	No	VIGILANCIA	Esc. 5
No	No	Sí	VIGILANCIA	Sin caso
No	Sí	Sí	ACTIVAR RESPALDO TÉRMICO	Esc. 2 y 3
Sí	Cualquiera	Cualquiera	ACTIVAR RESPALDO TÉRMICO	Esc. 4

Nota. Elaboración propia.

Pseudocódigo

El pseudocódigo se escribió antes del programa porque ayuda a ordenar los pasos antes de programar (Glorfeld y Palko, 1988). Cada opción del menú corresponde a un caso del Segun.

Figura 1

Pseudocódigo en PSeInt

```
Algoritmo HidroelectricaNino
  Definir opcion, dias, horas, n, i, conProblema
Como Entero
  Definir nivel, caudalMedio, demanda Como Real
  Definir caudal, altura, eficiencia, pNominal,
pBruta, pEfectiva, energia, factor Como Real
  Definir energiaTotal, caudalTotal, cobertura,
deficit, indiceAportes Como Real
  Definir region, nombre, buscado, estado, riesgo,
alerta Como Caracter
  Definir datosListos, repetido, encontrada, nino
Como Logico
  Dimension nombres[50], estados[50], energias[50],
caudales[50]
  Definir nombres, estados Como Caracter
  Definir energias, caudales Como Real
  n <- 0
  datosListos <- Falso
  opcion <- 0
  Mientras opcion <> 5 Hacer
    Escribir "1.Generales 2.Registrar 3.Consultar
4.Reporte 5.Salir"
    Leer opcion
    Segun opcion Hacer
      1:
        Leer region
        Repetir
          Leer dias
        Hasta Que dias >= 28 Y dias <= 31
        Repetir
          Leer horas
        Hasta Que horas >= 1 Y horas <= dias * 24
        Repetir
          Leer nivel
        Hasta Que nivel >= 0 Y nivel <= 100
        Repetir
          Leer caudalMedio, demanda
        Hasta Que caudalMedio > 0 Y demanda > 0
        datosListos <- Verdadero
      2:
        Si datosListos = Falso Entonces
          Escribir "Primero registre los datos
generales"
        SiNo
          Leer nombre
          repetido <- Falso
          Para i <- 1 Hasta n Hacer
            Si nombres[i] = nombre Entonces
              repetido <- Verdadero
            FinSi
          FinPara
          Si repetido Entonces
            Escribir "ERROR: esa central ya existe"
          SiNo
            Repetir
              Leer caudal, altura, eficiencia,
pNominal
            Hasta Que caudal > 0 Y altura > 0 Y
eficiencia > 0 Y eficiencia <= 1 Y pNominal > 0
            pBruta <- 9.81 * caudal * altura *
eficiencia / 1000
            Si pBruta > pNominal Entonces
              pEfectiva <- pNominal
            SiNo
              pEfectiva <- pBruta
            FinSi
            energia <- pEfectiva * horas / 1000
            factor <- pEfectiva * horas / (pNominal
* dias * 24) * 100
            n <- n + 1
            nombres[n] <- nombre
            estados[n] <- estado
            energias[n] <- energia
            caudales[n] <- caudal
          FinSi
        FinSi
      3:
        Leer buscado
        encontrada <- Falso
        Para i <- 1 Hasta n Hacer
          Si nombres[i] = buscado Entonces
            encontrada <- Verdadero
            Escribir energias[i], " GWh ",
estados[i]
          FinSi
        FinPara
        Si encontrada = Falso Entonces
          Escribir "No existe una central con ese
nombre"
        FinSi
      4:
        energiaTotal <- 0
        caudalTotal <- 0
        conProblema <- 0
        Para i <- 1 Hasta n Hacer
          Escribir nombres[i], " ", energias[i], "
GWh ", estados[i]
          energiaTotal <- energiaTotal + energias[i]
          caudalTotal <- caudalTotal + caudales[i]
          Si estados[i] <> "ADECUADA" Entonces
            conProblema <- conProblema + 1
          FinSi
        FinPara
        cobertura <- energiaTotal / demanda * 100
        Si energiaTotal < demanda Entonces
          deficit <- demanda - energiaTotal
        SiNo
          deficit <- 0
        FinSi
        indiceAportes <- caudalTotal / caudalMedio
        nino <- indiceAportes < 0.80
        Si nivel > 70 Y cobertura >= 100 Entonces
          riesgo <- "NORMAL"
        SiNo
          Si nivel > 50 Entonces
            riesgo <- "SEGUIMIENTO"
          SiNo
            Si nivel > 30 Entonces
              riesgo <- "ALERTA"
            SiNo
              riesgo <- "CRITICO"
            FinSi
          FinSi
        FinSi
        Si riesgo = "CRITICO" O (deficit > 0 Y nino)
Entonces
          alerta <- "ACTIVAR RESPALDO TERMICO"
        SiNo
          Si deficit > 0 O nino Entonces
            alerta <- "VIGILANCIA"
          SiNo
            alerta <- "OPERACION NORMAL"
          FinSi
        FinSi
        Escribir energiaTotal, cobertura, deficit,
riesgo, alerta
      5:
        Escribir "Fin del programa"
```

```
            Si pBruta > pNominal Entonces                    De Otro Modo:
              estado <- "LIMITADA"                             Escribir "ERROR: opcion no valida"
            SiNo                                             FinSegun
              Si factor < 30 Entonces                      FinMientras
                estado <- "SUBUTILIZADA"                 FinAlgoritmo
              SiNo
                Si eficiencia < 0.85 Entonces
                  estado <- "EFICIENCIA BAJA"
                SiNo
                  estado <- "ADECUADA"
                FinSi
              FinSi
            FinSi
```

Nota. Elaboración propia.

Programa en Python

El programa sigue el mismo orden del pseudocódigo: un ciclo while muestra el menú hasta que el usuario elige salir, y cada opción se atiende con if-elif.

Figura 2

Programa en Python (generacion_hidro_nino.py)

```
# ABP Fase 2 - Generacion hidroelectrica durante El Nino
# Programa con menu, condicionales y ciclos

# Constantes
GRAVEDAD = 9.81
MAX_CENTRALES = 50

# Listas donde se guardan las centrales
nombres = []
potencias = []
energias = []
factores = []
estados = []
caudales = []

datos_listos = False
opcion = 0

while opcion != 5:
    print("")
    print("1. Registrar datos generales")
    print("2. Registrar central")
    print("3. Consultar central")
    print("4. Generar reporte")
    print("5. Salir")
    opcion = int(input("Opcion: "))
    while opcion < 1 or opcion > 5:
        print("ERROR: la opcion debe estar entre 1 y 5")
        opcion = int(input("Opcion: "))

    if opcion == 1:
        region = input("Nombre de la region: ")
        dias = int(input("Dias del mes (28 a 31): "))
        while dias < 28 or dias > 31:
            print("ERROR: los dias deben estar entre 28 y 31")
            dias = int(input("Dias del mes (28 a 31): "))
        horas = int(input("Horas de operacion en el mes: "))
        while horas < 1 or horas > dias * 24:
            print("ERROR: las horas deben estar entre 1 y", dias * 24)
            horas = int(input("Horas de operacion en el mes: "))
        nivel = float(input("Nivel del embalse (0 a 100 %): "))
        while nivel < 0 or nivel > 100:
            print("ERROR: el nivel debe estar entre 0 y 100")
            nivel = float(input("Nivel del embalse (0 a 100 %): "))
        caudal_medio = float(input("Caudal medio historico (m3/s): "))
        while caudal_medio <= 0:
            print("ERROR: el caudal medio debe ser mayor que 0")
            caudal_medio = float(input("Caudal medio historico (m3/s): "))
        demanda = float(input("Demanda del mes (GWh): "))
        while demanda <= 0:
            print("ERROR: la demanda debe ser mayor que 0")
```

```
            demanda = float(input("Demanda del mes (GWh): "))
        datos_listos = True
        print("Datos generales registrados")

    elif opcion == 2:
        if datos_listos == False:
            print("Primero registre los datos generales (opcion 1)")
        elif len(nombres) >= MAX_CENTRALES:
            print("ERROR: ya hay", MAX_CENTRALES, "centrales registradas")
        else:
            nombre = input("Nombre de la central: ").upper()
            repetido = False
            for i in range(len(nombres)):
                if nombres[i] == nombre:
                    repetido = True
            if repetido == True:
                print("ERROR: esa central ya existe")
            else:
                caudal = float(input("Caudal turbinado (m3/s): "))
                while caudal <= 0:
                    print("ERROR: el caudal debe ser mayor que 0")
                    caudal = float(input("Caudal turbinado (m3/s): "))
                altura = float(input("Altura neta (m): "))
                while altura <= 0:
                    print("ERROR: la altura debe ser mayor que 0")
                    altura = float(input("Altura neta (m): "))
                eficiencia = float(input("Eficiencia (0 a 1): "))
                while eficiencia <= 0 or eficiencia > 1:
                    print("ERROR: la eficiencia debe estar entre 0 y 1")
                    eficiencia = float(input("Eficiencia (0 a 1): "))
                p_nominal = float(input("Potencia nominal (MW): "))
                while p_nominal <= 0:
                    print("ERROR: la potencia nominal debe ser mayor que 0")
                    p_nominal = float(input("Potencia nominal (MW): "))

                # Calculos de la central
                p_bruta = GRAVEDAD * caudal * altura * eficiencia / 1000   # MW
                if p_bruta > p_nominal:
                    p_efectiva = p_nominal
                else:
                    p_efectiva = p_bruta
                energia = p_efectiva * horas / 1000                        # GWh
                factor = p_efectiva * horas / (p_nominal * dias * 24) * 100

                # Estado de la instalacion
                if p_bruta > p_nominal:
                    estado = "LIMITADA"
                elif factor < 30:
                    estado = "SUBUTILIZADA"
                elif eficiencia < 0.85:
                    estado = "EFICIENCIA BAJA"
                else:
                    estado = "ADECUADA"

                nombres.append(nombre)
                caudales.append(caudal)
                potencias.append(p_efectiva)
                energias.append(energia)
                factores.append(factor)
                estados.append(estado)
                print("Central", nombre, "registrada:", estado)

    elif opcion == 3:
        if len(nombres) == 0:
            print("No hay centrales registradas")
        else:
            buscado = input("Nombre de la central a consultar: ").upper()
            encontrada = False
            for i in range(len(nombres)):
                if nombres[i] == buscado:
                    encontrada = True
                    print("Potencia efectiva:", round(potencias[i], 2), "MW")
                    print("Energia del mes:", round(energias[i], 2), "GWh")
                    print("Factor de planta:", round(factores[i], 1), "%")
                    print("Estado:", estados[i])
            if encontrada == False:
                print("No existe una central con ese nombre")

    elif opcion == 4:
```

```
        if len(nombres) == 0:
            print("No hay centrales registradas")
        else:
            energia_total = 0
            caudal_total = 0
            con_problema = 0
            print("REPORTE DE LA REGION", region)
            for i in range(len(nombres)):
                print(nombres[i], round(energias[i], 2), "GWh", estados[i])
                energia_total = energia_total + energias[i]
                caudal_total = caudal_total + caudales[i]
                if estados[i] != "ADECUADA":
                    con_problema = con_problema + 1

            cobertura = energia_total / demanda * 100
            if energia_total < demanda:
                deficit = demanda - energia_total
            else:
                deficit = 0
            indice_aportes = caudal_total / caudal_medio
            if indice_aportes < 0.80:
                nino = True
            else:
                nino = False

            # Clasificacion del riesgo
            if nivel > 70 and cobertura >= 100:
                riesgo = "NORMAL"
            elif nivel > 50:
                riesgo = "SEGUIMIENTO"
            elif nivel > 30:
                riesgo = "ALERTA"
            else:
                riesgo = "CRITICO"

            # Alerta operativa
            if riesgo == "CRITICO" or (deficit > 0 and nino == True):
                alerta = "ACTIVAR RESPALDO TERMICO"
            elif deficit > 0 or nino == True:
                alerta = "VIGILANCIA"
            else:
                alerta = "OPERACION NORMAL"

            print("Energia total:", round(energia_total, 2), "GWh")
            print("Cobertura de la demanda:", round(cobertura, 1), "%")
            print("Deficit:", round(deficit, 2), "GWh")
            print("Indice de aportes:", round(indice_aportes, 2))
            print("Centrales con problema:", con_problema, "de", len(nombres))
            print("Riesgo:", riesgo)
            print("Alerta:", alerta)

print("Fin del programa")
```

Nota. Elaboración propia.

Pruebas de escritorio

La prueba de escritorio consiste en seguir el programa paso a paso con datos conocidos (Lister et al., 2004). Se usaron tres centrales de ejemplo: CH-1 (H = 175 m, η = 0,90, P_n = 290 MW), CH-2 (150 m, 0,88, 165 MW) y CH-3 (120 m, 0,86, 60 MW), con 30 días, caudal medio de 385 m³/s y demanda de 240 GWh. Los escenarios 1 a 4 representan un año normal y tres niveles de El Niño; el escenario 5 repite el 1 con demanda de 300 GWh y eficiencia de CH-3 de 0,75. Todos los resultados se obtuvieron ejecutando el programa.

Tabla 8

Matriz de pruebas de validación de entradas

Caso	Entrada	Salida esperada	Resultado
V-01	Opción 7	ERROR: la opción debe estar entre 1 y 5	Aprobado
V-02	Opción 4 sin centrales	No hay centrales registradas	Aprobado
V-03	Opción 2 sin datos generales	Primero registre los datos generales	Aprobado
V-04	Días = 32	ERROR: los días deben estar entre 28 y 31	Aprobado
V-05	Horas = 800 con 30 días	ERROR: las horas deben estar entre 1 y 720	Aprobado
V-06	Nivel = 120 %	ERROR: el nivel debe estar entre 0 y 100	Aprobado
V-07	Caudal medio = 0	ERROR: debe ser mayor que 0	Aprobado
V-08	Caudal = −5	ERROR: el caudal debe ser mayor que 0	Aprobado
V-09	Eficiencia = 1,5	ERROR: la eficiencia debe estar entre 0 y 1	Aprobado
V-10	Nombre «ch-1» ya registrado	ERROR: esa central ya existe	Aprobado
V-11	Consultar «CH-9»	No existe una central con ese nombre	Aprobado
V-12	Consultar «ch-1» (Esc. 1)	290,0 MW; 145,0 GWh; 69,4 %; LIMITADA	Aprobado

Nota. Después de cada error el programa vuelve a pedir el dato. El programa convierte los nombres a mayúsculas, por eso «ch-1» y «CH-1» se consideran iguales. Elaboración propia.

Tabla 9

Matriz de resultados del reporte

Dato o resultado	Esc. 1	Esc. 2	Esc. 3	Esc. 4	Esc. 5
Horas de operación	500	520	560	600	500
Nivel del embalse (%)	85	60	40	25	85
Caudales CH-1 / CH-2 / CH-3 (m^3/s)	200 / 130 / 55	124 / 80,6 / 34,1	76 / 49,4 / 20,9	56 / 36,4 / 15,4	200 / 130 / 55
Energía total (GWh)	255,34	171,85	113,43	89,55	251,78
Cobertura (%)	106,4	71,6	47,3	37,3	83,9
Déficit (GWh)	0	68,15	126,57	150,45	48,22
Índice de aportes	1,00	0,62	0,38	0,28	1,00
Centrales con problema	2 de 3	0 de 3	1 de 3	3 de 3	3 de 3
Riesgo	NORMAL	SEGUIMIENTO	ALERTA	CRÍTICO	SEGUIMIENTO
Alerta	NORMAL	RESPALDO	RESPALDO	RESPALDO	VIGILANCIA
¿Coincide con lo esperado?	Sí	Sí	Sí	Sí	Sí

Nota. RESPALDO abrevia «ACTIVAR RESPALDO TÉRMICO». Elaboración propia.

Tabla 10

Prueba de escritorio del ciclo for del reporte (Escenario 5)

i	nombres[i]	energias[i] (GWh)	estados[i]	energia_total	con_problema
0	CH-1	145,00	LIMITADA	145,00	1
1	CH-2	82,50	LIMITADA	227,50	2
2	CH-3	24,28	EFICIENCIA BAJA	251,78	3

Nota. En Python las listas empiezan en la posición 0. Elaboración propia.

Análisis de resultados

Los resultados muestran que, a medida que El Niño se hace más fuerte, baja la energía generada y aumenta el déficit, hasta llegar a riesgo CRÍTICO en el escenario 4. El estado de cada central permite ver problemas de la instalación: en un año normal dos centrales quedan limitadas

por su potencia nominal, en El Niño fuerte las tres quedan subutilizadas y en el escenario 5 CH-3 aparece con eficiencia baja. Las doce pruebas de validación fueron aprobadas.

Conclusiones

En la Fase 2 se construyó un programa sencillo en Python que usa el ciclo while para repetir el menú y validar datos, el ciclo for para buscar, consultar y sumar, y condicionales para aplicar diez reglas de negocio. Las matrices de decisiones y las pruebas de escritorio comprobaron que cada caso da el resultado esperado. Además de clasificar el riesgo de la región, el programa indica qué central tiene problemas, lo que responde al eje problematizador. Como limitación, si se escribe una letra donde se espera un número el programa se detiene; esto se puede mejorar en la siguiente fase.

Referencias

Cramton, P., y Stoft, S. (2007). Colombia firm energy market. En *Proceedings of the 40th Annual Hawaii International Conference on System Sciences (HICSS'07)* (p. 124). IEEE. https://doi.org/10.1109/HICSS.2007.133

Ehsan, H., Rehmat, A. P., y Cardella, M. E. (2020). Computational thinking embedded in engineering design: Capturing computational thinking of children in an informal engineering design activity. *International Journal of Technology and Design Education, 31*(3), 441–464. https://doi.org/10.1007/s10798-020-09562-5

Fangohr, H. (2004). A comparison of C, MATLAB, and Python as teaching languages in engineering. En M. Bubak, G. D. van Albada, P. M. A. Sloot y J. Dongarra (Eds.), *Computational Science – ICCS 2004* (Lecture Notes in Computer Science, Vol. 3039, pp. 1210–1217). Springer. https://doi.org/10.1007/978-3-540-25944-2_157

Glorfeld, L. W., y Palko, J. (1988). A comparison of novice algorithm composition performance using flowcharting or pseudocode as the design tool. *Journal of Research on Computing in Education, 21*(1), 82–96. https://doi.org/10.1080/08886504.1988.10781862

Gómez-Dueñas, S., Bateman, A., y Santos Granados, G. R. (2024). Untangling the implications of climate-forcing and human-induced drivers in streamflow variability: The Magdalena River, Colombia. *Hydrological Sciences Journal, 69*(8), 1046–1059. https://doi.org/10.1080/02626667.2024.2352517

Hoseinzadeh, T., Shourian, M., y Yazdi, J. (2020). Optimum design and operation of a hydropower reservoir considering uncertainty of inflow. *Journal of Hydroinformatics, 22*(6), 1452–1467. https://doi.org/10.2166/hydro.2020.044

Kunkle, W. M., y Allen, R. B. (2016). The impact of different teaching approaches and languages on student learning of introductory programming concepts. *ACM Transactions on Computing Education, 16*(1), 1–26. https://doi.org/10.1145/2785807

Lister, R., Adams, E. S., Fitzgerald, S., Fone, W., Hamer, J., Lindholm, M., McCartney, R., Moström, J. E., Sanders, K., Seppälä, O., Simon, B., y Thomas, L. (2004). A multi-national study of reading and tracing skills in novice programmers. *ACM SIGCSE Bulletin, 36*(4), 119–150. https://doi.org/10.1145/1041624.1041673

Medina, S., y Moreno, J. (2007). Risk evaluation in Colombian electricity market using fuzzy logic. *Energy Economics, 29*(5), 999–1009. https://doi.org/10.1016/j.eneco.2006.02.008

Moreno Chuquen, R., y Cantillo Luna, S. A. (2020). Análisis del riesgo de la cobertura de energía firme en Colombia a mediano plazo. *Tecnura, 24*(64), 23–33. https://doi.org/10.14483/22487638.16487

Poveda, G., Jaramillo, A., Gil, M. M., Quiceño, N., y Mantilla, R. I. (2001). Seasonality in ENSO-related precipitation, river discharges, soil moisture, and vegetation index in Colombia. *Water Resources Research, 37*(8), 2169–2178. https://doi.org/10.1029/2000WR900395

Qian, Y., y Lehman, J. (2017). Students' misconceptions and other difficulties in introductory programming: A literature review. *ACM Transactions on Computing Education, 18*(1), 1–24. https://doi.org/10.1145/3077618

Restrepo-Trujillo, J., Moreno-Chuquen, R., Jiménez-García, F. N., Flores, W. C., y Chamorro, H. R. (2022). Scenario analysis of an electric power system in Colombia considering the El Niño phenomenon and the inclusion of renewable energies. *Energies, 15*(18), Artículo 6690. https://doi.org/10.3390/en15186690

Sritram, P., y Suntivarakorn, R. (2017). Comparative study of small hydropower turbine efficiency at low head water. *Energy Procedia, 138*, 646–650. https://doi.org/10.1016/j.egypro.2017.10.181

Tayyeh, H. K., y Mohammed, R. (2024). Vulnerability and resilience of hydropower generation under climate change scenarios: Haditha dam reservoir case study. *Applied Energy, 366*, Artículo 123308. https://doi.org/10.1016/j.apenergy.2024.123308